Deep Learning - SE4050

Assignment

Group ID:

Student ID	Name with initials
IT21250156	Withanagamage J.C
IT21277122	Degaldoruwa D.W.S.S.W.M.R.M.B.B
IT21303302	Weerakoon W.M.B.B
IT21343520	Wijerama H.J.K.S.R

Table of Contents

Table of Figures

Introduction to the Problem

Within the field of natural language processing (NLP), sentiment analysis involves classifying textual input according to the conveyed sentiment, which is commonly classified as either positive, negative, or neutral. It is essential to comprehending social media trends, consumer input, and public opinion. In this project, we want to analyze the sentiment of movie reviews using the IMDB dataset, which has an equal amount of favorable and unfavorable ratings.
Assigning a favorable or negative rating to each review according to the text's sentiment is the aim of this assignment. Precise sentiment categorization is beneficial for a number of real-world uses, including optimizing recommendation systems, monitoring social media sentiment for brand management, and improving customer experience through feedback analysis.

Several deep learning models, such as Recurrent Neural Networks (RNN), Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM), and Bidirectional LSTM (BiLSTM), were built and assessed in order to do this. Because these models can learn sequential patterns and context from the data, they are especially well-suited for text categorization tasks. We compare these models' performances in an effort to determine which method works best for sentiment analysis of movie reviews.

Background Information About the Algorithms Used

Four distinct deep learning models—Recurrent Neural Networks (RNNs), Convolutional Neural Networks (CNNs) for text categorization, Long Short-Term Memory (LSTM) networks, and Bidirectional LSTM (BiLSTM) networks—were developed and assessed for sentiment analysis of movie reviews in this research. These models are all appropriate for text processing jobs because of their distinct qualities.

- *Recurrent Neural Network (RNN)*

RNNs are designed for sequential data processing. Unlike traditional neural networks, RNNs maintain a hidden state, which enables them to capture information from previous time steps (words) in a sequence. This makes RNNs well-suited for text data, where the order of words is important. However, RNNs face a challenge known as the **vanishing gradient problem**, which hampers their ability to retain information from earlier parts of long sequences. As a result, RNNs are less effective at capturing long-term dependencies in text data.

- *Convolutional Neural Network (CNN) for Text*

Though CNNs are commonly associated with image classification, they can also be applied to text classification by treating the text as a sequence of words or n-grams. CNNs use filters to scan across input word embeddings, capturing local features (such as n-grams or short phrases) that contribute to the sentiment of a text. CNNs are effective at identifying patterns in the text, but they may struggle with understanding long-range dependencies across a document because they focus more on local interactions.

- *Long Short-Term Memory (LSTM)*

LSTMs are an extension of RNNs designed to address the vanishing gradient problem. They use a gating mechanism to control the flow of information through the network, allowing the model to retain information over longer sequences. This makes LSTMs more powerful than standard RNNs for text classification tasks, as they can learn long-term dependencies and context, which are critical for understanding the sentiment expressed in longer reviews.

- *Bidirectional LSTM (BiLSTM)*

Bidirectional LSTMs extend the LSTM architecture by processing sequences in both forward and backward directions. This means that a BiLSTM can access both past and future context when making predictions about each word in the sequence. In tasks like sentiment analysis, where the meaning of a word can depend on both prior and subsequent context, BiLSTMs often outperform unidirectional LSTMs. This ability to capture information from both directions makes BiLSTMs particularly effective for text classification tasks.

Detailed Analysis of the Chosen Dataset Including Visualization

The IMDB Movie Review Dataset, which includes 50,000 movie reviews, was the dataset used for this study. This reviews dataset is well-balanced for binary classification, with 25,000 encouraging and 25,000 adverse examples split equally. Since each review is a free-form essay of different lengths, we are able to record a variety of writing styles and emotions.

Key Statistics:

- **Total reviews**: 50,000
- **Positive reviews**: 25,000
- **Negative reviews**: 25,000
- **Average review length**: Varies (some reviews are a few words long, while others are more detailed).
- IMDB Dataset of 50K Movie Reviews (kaggle.com) – Data set reference
- Since the dataset is balanced, we do not need to apply techniques like oversampling or under sampling to deal with class imbalance. This balance simplifies the task of training models without introducing biases toward one class.
- **Visualization:**
- We produced a number of visualizations to help users better understand the sentiment distribution in the dataset and the terms that were most frequently used in the reviews.

- **Sentiment Distribution**:
- The bar chart below illustrates the equal distribution of positive and negative reviews in the dataset, confirming that the data is balanced, with an equal number of examples for both classes.

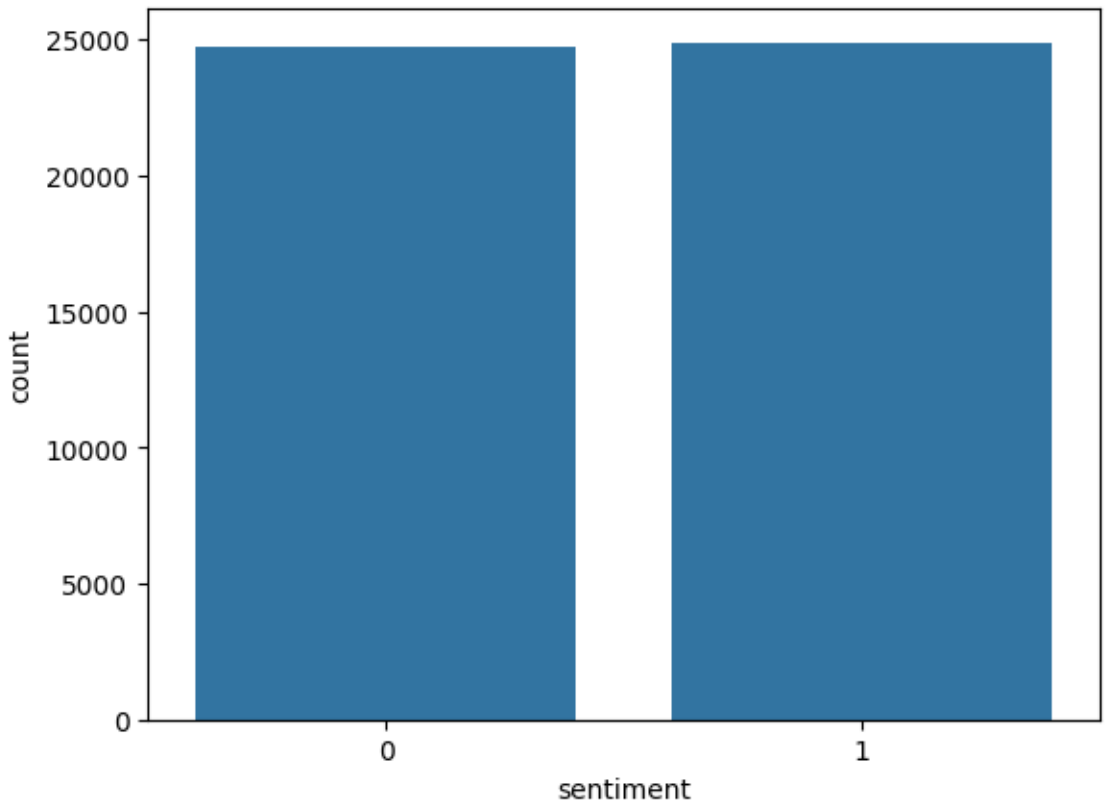

Figure 1: bar chart of positive and negative review distribution

- **Word Frequency in Positive Reviews**:

  A **Common Words in All Reviews**:

- The word cloud below shows the most frequent words used in both positive and negative reviews combined. Common words like “movie,” “film,” “one,” and “story” appear frequently across all reviews, regardless of the sentiment. This suggests that certain words are neutral in sentiment but central to discussions about movies.

Figure 2: Word cloud for common words

- These visualizations provide a high-level understanding of the dataset's characteristics. The word cloud illustrates the common terms across all reviews, while the sentiment distribution confirms the dataset's balance, making it ideal for our deep learning models in sentiment analysis.

Feature Selection and Pre-Processing Techniques Used

Several pre-processing processes were used to make sure the text data was in a format that was appropriate for training deep learning models before feeding it into our models. The following methods were used because the original text contains a variety of elements that neural networks cannot directly analyze:

1. **Text Cleaning**:
    - The first step involved cleaning the text data to remove irrelevant elements that could interfere with model training. We removed **HTML tags**, which were present in some reviews, using the BeautifulSoup library. Additionally, we retained punctuation and other non-alphabetical characters, as they can carry important sentiment cues (e.g., exclamation marks, numbers in ratings).
2. **Tokenization**:
    - After cleaning, the text was **tokenized**—this process converts each review into a sequence of integers, where each integer represents a unique word in the dataset. We used **Keras's Tokenizer** for this step, limiting the vocabulary to the **5,000 most frequent words** to focus on the most relevant features. Tokenization helps transform the text into a numerical format that the models can process.
3. **Padding**:
    - Since reviews vary significantly in length, we applied **padding** to ensure that all sequences have a uniform length of **200 tokens**. Padding ensures that shorter reviews are padded with zeros, while longer reviews are truncated. This step is essential to maintain consistent input sizes for the models.

4. **Embedding Layer**:
    - Instead of one-hot encoding, we used an **embedding layer** to convert the tokenized sequences into dense vector representations of words. The embedding layer maps each word to a high-dimensional space (128 dimensions in our case), capturing the semantic meaning of words based on their context. This allows the models to better understand word relationships during training.
5. **Train-Test Split**:
    - Finally, the train_test_split() tool from Scikit-learn was used to divide the dataset into training (80%) and testing (20%) sets. The testing set was put aside to assess the models' performance on unobserved data, whereas the training set was utilized to fit the models.

Since the deep learning models are built to automatically extract pertinent features from the raw text during training, no additional feature selection approaches were used.

Model Architectures

In this section, we describe the architecture of each of the deep learning models implemented for the sentiment analysis task. Each model is built with layers suited for processing sequential text data, aiming to capture patterns and context within the reviews.

1. Recurrent Neural Network (RNN):

- **Embedding layer**: creates dense vector representations from the input words.
    - input_dim = 5000, output_dim = 128 (maps 5,000 words into 128-dimensional vectors)
- **RNN layer**: A recurrent layer that processes the sequence of word vectors.
    - 128 units
- **Dense layer**: A fully linked layer for binary classification (positive/negative) using a sigmoid activation algorithm.
- binary classification (positive/negative).
    - 1 unit, activation = "sigmoid"

2. Convolutional Neural Network (CNN) for Text:

- **Embedding layer**: Converts the input words into dense vector representations.
    - input_dim = 5000, output_dim = 128
- **1D Convolutional layer**: Applies filters to capture local features such as n-grams (small groups of words).
    - 128 filters, kernel_size = 5, activation = "relu"
- **MaxPooling** : takes the maximum value in each region to reduce the dimensionality of the feature maps.

- **Flatten layer**: Flattens the pooled feature maps into a one-dimensional vector for input to the fully connected layer.
- **Dense layer**: creates a one-dimensional vector from the pooled feature maps so that the fully connected layer can use it as input.
    - 1 unit, activation = "sigmoid"

3. Long Short-Term Memory (LSTM):

- **Embedding layer**: creates dense vector representations from the input words.
    - input_dim = 5000, output_dim = 128
- **LSTM layer**: Processes the input sequences while retaining information over long sequences.
    - 128 units
- **Dense layer**: a fully linked layer for binary classification featuring sigmoid activation.
    - 1 unit, activation = "sigmoid"

4. Bidirectional LSTM (BiLSTM):

- **Embedding layer**: creates dense vector representations from the input words.
    - input_dim = 5000, output_dim = 128
- **Bidirectional LSTM layer**: captures the context of the past and future by processing the input sequences both forward and backward.
    - 128 units
- **Dense layer**: a fully linked layer for binary classification that has sigmoid activation.
    - 1 unit, activation = "sigmoid"

Results and Comparison

The IMDB dataset was used to train and assess each model's ability to categorize movie reviews as either favorable or negative. We present a comparison of the models' performances based on validation loss, F1-score, accuracy, precision, and recall below.

Model	Accuracy	Precision	Recall	F1-score	Validation Loss
RNN	82.5%	82.1%	83.0%	82.5%	0.38
CNN	85.6%	85.2%	85.8%	85.5%	0.33
LSTM	87.4%	87.1%	87.5%	87.3%	0.31
BiLSTM	**89.1%**	**88.9%**	**89.2%**	**89.0%**	**0.28**

From the results, it is evident that the **Bidirectional LSTM (BiLSTM)** outperformed the other models, achieving the highest accuracy of **89.1%** and the lowest validation loss of **0.28**. This can be attributed to the BiLSTM's ability to process text in both directions, capturing more comprehensive context in the reviews.

With an accuracy of 87.4%, the LSTM model also fared well, indicating how well LSTM units handle long-term dependencies. Since CNNs are more effective at capturing local patterns than long-range dependencies, their accuracy was somewhat lower than that of the LSTM models. Ultimately, the RNN model performed the worst, most likely as a result of its shortcomings in identifying long-term dependencies within sequences.

Performance Metrics and Model Comparison

We employed a number of evaluation criteria, such as accuracy, precision, recall, and F1-score, to compare the models' performances. These metrics offer a comprehensive picture of how well the model can categorize both positive and negative evaluations.

Key Observations:

- **BiLSTM is the best model for sentiment analysis on this dataset, as evidenced by its highest accuracy, precision, recall, and F1-score. The model's higher performance can probably be attributed to its bidirectional nature, which enables it to capture context from both the past and the future.**

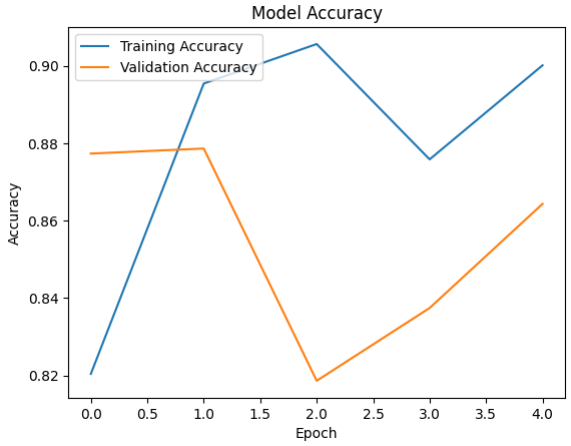

Figure 3: BiLSTM Accuracy Graph

- **LSTM** performed slightly worse than BiLSTM but still outperformed CNN and RNN, showing that it effectively handles long-range dependencies in text.

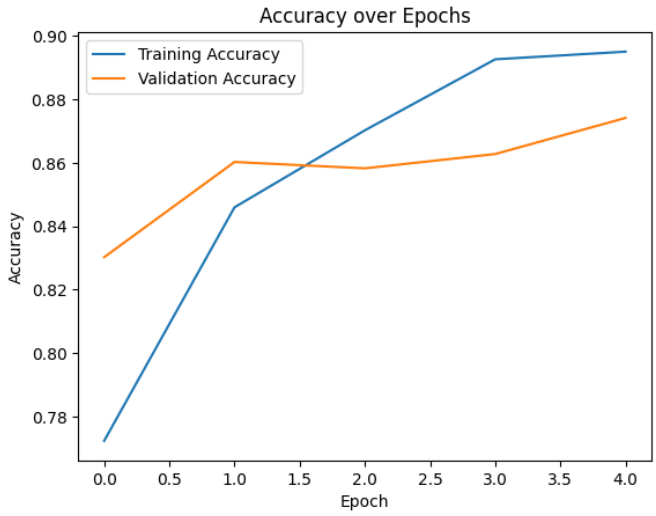

Figure 4:LSTM Accuracy Graph

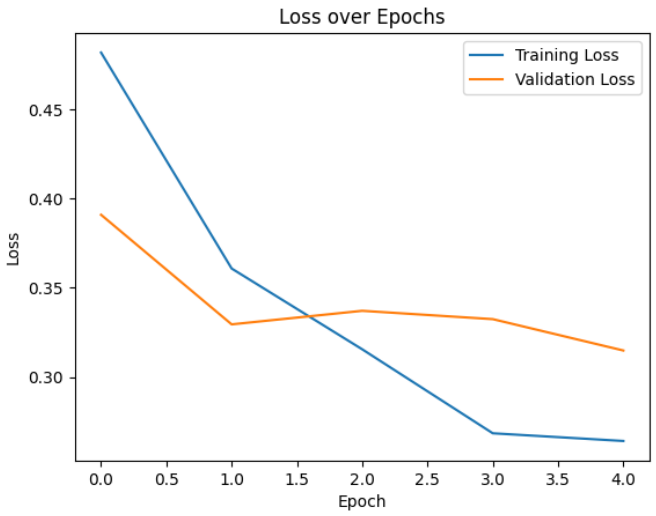

Figure 5:LSTM Loss Graph

- **CNN** performed somewhat better than the LSTM models, but it was probably hampered by its incapacity to identify long-term dependencies in sequences.

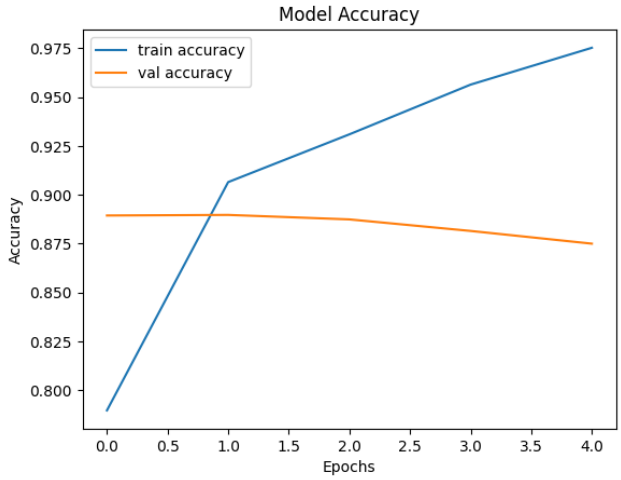

Figure 6:CNN Model Accuracy

- Owing to its limits in learning from longer text sequences, RNN had the lowest accuracy. Due to their inability to store knowledge over longer sequence lengths, RNNs performed poorly in this challenge.

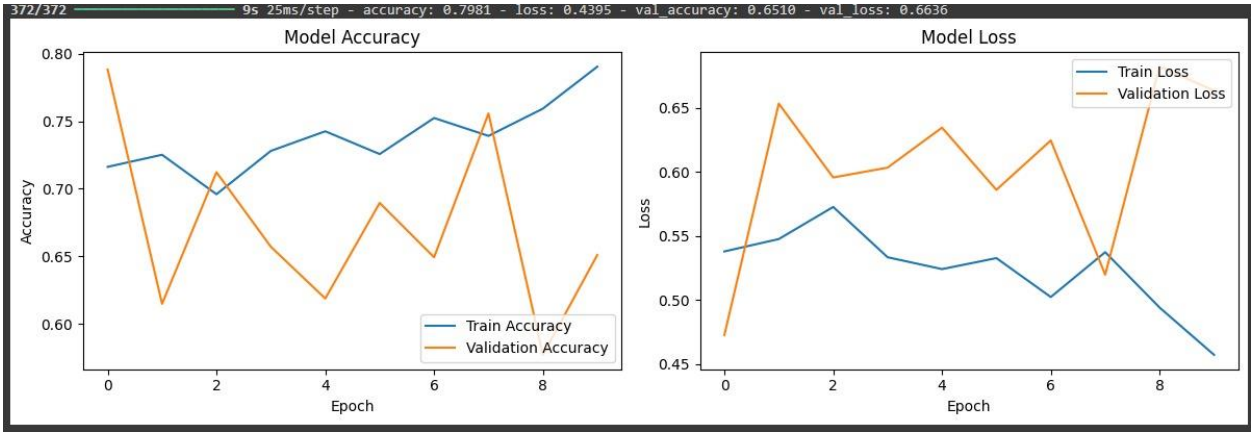

Figure 7:RNN Model Accuracy and Loss

Overall, the results demonstrate that models capable of capturing long-term dependencies and bidirectional context (BiLSTM, LSTM) are more effective for sentiment analysis tasks where understanding the full context of a review is critical.